



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра алгоритмических языков

Лебедев Андрей Алексеевич

**Методы обучения с подкреплением для адаптации
больших языковых моделей к задаче
извлечения именованных сущностей на русском языке**

Курсовая работа

Научный руководитель:
Канд. физ.-мат. наук
М.М. Тихомиров

Москва, 2026

Аннотация

Методы обучения с подкреплением для адаптации
больших языковых моделей к задаче
извлечения именованных сущностей на русском языке

Лебедев Андрей Алексеевич

В работе исследуются методы обучения с подкреплением для адаптации больших языковых моделей к задаче извлечения именованных сущностей на русском языке. Проведено сравнение Supervised Fine-Tuning (SFT) и обучения методом Group Relative Policy Optimization (GRPO) для модели Qwen3-4B на наборе данных NEREL. Реализован полный экспериментальный пайплайн, включающий подготовку данных, обучение моделей и комплексную оценку качества. Эксперименты показывают, что GRPO позволяет существенно улучшить качество извлечения сущностей и обеспечивает более устойчивый перенос навыка на различные NER-бенчмарки.

Содержание

1	Введение	4
2	Обзор существующих методов	6
2.1	Задача извлечения именованных сущностей	6
2.2	Большие языковые модели в задаче NER	7
2.3	Обучение с учителем	7
2.4	Обучение с подкреплением	8
2.5	Group Relative Policy Optimization	10
2.6	DAPO	12
2.7	Другие модификации GRPO	13
3	Постановка задачи	16
4	Практическая часть	18
4.1	Общая схема экспериментов	18
4.2	Подготовка данных и формирование запросов	19
4.3	Обучение методом SFT	20
4.4	Обучение методом GRPO	21
4.5	Оценка качества на задаче NER	23
4.6	Оценка с использованием LLMTF Open	23
4.7	Результаты экспериментов	25
4.7.1	Проведенные серии экспериментов	25
4.7.2	Итоговое сравнение на LLMTF	26
4.7.3	Анализ полученных результатов	26
5	Инструментальные средства	29
6	Заключение	30
	Список литературы	31

1 Введение

Большие языковые модели в последние годы стали одним из основных инструментов обработки естественного языка. Современные LLM способны решать широкий круг задач: отвечать на вопросы, выполнять перевод, суммаризацию, программирование и анализ текста. При этом практическое применение таких моделей часто требует дополнительной адаптации к специализированным задачам и предметным областям.

Одной из таких задач является извлечение именованных сущностей (Named Entity Recognition, NER). Задача NER состоит в выделении в тексте упоминаний объектов заданных типов: персон, организаций, географических объектов, дат и других категорий. Извлечение сущностей используется в информационном поиске, анализе документов, построении баз знаний и системах извлечения информации.

В классической постановке NER решается как задача классификации токенов. Однако развитие инструктивных LLM привело к распространению подхода, в котором модель получает текст и инструкцию, а затем формирует структурированный ответ со списком найденных сущностей. Такой подход позволяет использовать одну и ту же языковую модель для разных задач без изменения архитектуры выходного слоя.

Наиболее распространенным способом адаптации LLM к новым задачам остается обучение с учителем. Несмотря на высокую эффективность, этот подход имеет существенное ограничение: узкая специализация модели может сопровождаться ухудшением других способностей. Для больших языковых моделей эта проблема существенна, поскольку практическая ценность LLM определяется не только качеством на одной задаче, но и сохранением универсальных навыков.

В последние годы активно развиваются методы обучения с подкреплением для LLM. Появление задач с автоматически проверяемой наградой привело к распространению методов RL для обучения математическим рассуждениям, программированию и структурированной генерации. Одним из наиболее заметных методов этого направления стал Group Relative Policy Optimization (GRPO), предложенный в работе DeepSeekMath.

Настоящая работа посвящена исследованию методов обучения с подкреплением для адаптации LLM к задаче извлечения именованных сущностей на русском языке. В качестве базовой модели используется Qwen3-4B, а обучение проводится на наборе данных NEREL. В работе сравниваются два подхода: SFT (Supervised Fine-Tuning) и обучение

методом GRPO.

Цель работы состоит в исследовании того, позволяет ли обучение с подкреплением улучшить качество извлечения сущностей без существенного ухудшения других способностей модели. Для этого проводится комплексная оценка моделей не только на NER-бенчмарках, но и на широком наборе задач с использованием фреймворка LLMTF Open.

2 Обзор существующих методов

В данной работе рассматривается задача адаптации больших языковых моделей к извлечению именованных сущностей. Эта задача находится на пересечении нескольких направлений: классического извлечения сущностей, решения задач обработки естественного языка, дообучения моделей на инструкциях и обучения с подкреплением с автоматически проверяемой наградой. В этом разделе рассматриваются основные подходы, на которые опирается настоящая работа.

2.1 Задача извлечения именованных сущностей

Извлечение именованных сущностей (Named Entity Recognition, NER) является одной из базовых задач обработки естественного языка. Ее цель состоит в выделении в тексте фрагментов, соответствующих объектам заранее заданных типов: персонам, организациям, географическим объектам, датам, событиям и другим категориям. В классической постановке задача NER обычно формулируется как задача последовательной разметки: каждому токenu текста сопоставляется метка, указывающая, является ли он частью сущности и к какому типу эта сущность относится.

Для русского языка задача извлечения сущностей имеет ряд дополнительных сложностей. Они связаны с развитой морфологией, свободным порядком слов, неоднозначностью границ сущностей, а также с большим числом вариантов написания собственных имен. Кроме того, в прикладных задачах часто требуется извлекать не только плоские, но и вложенные сущности. Например, внутри названия организации может встречаться географическое название, а внутри названия события — дата или имя участника. Такая постановка называется *nested NER* и является более сложной, чем стандартная токен-классификация, поскольку один и тот же фрагмент текста может входить сразу в несколько сущностей.

NEREL [2] представляет собой русскоязычный набор данных для извлечения именованных сущностей, отношений и событий. Одной из его особенностей является поддержка вложенных сущностей, при которой один фрагмент текста может одновременно входить в состав нескольких сущностей разных типов. Такая структура разметки делает задачу существенно сложнее по сравнению с классическим NER и позволяет оценивать способность модели извлекать более сложные семантические структуры текста. В на-

стоящей работе NEREL используется в качестве основного набора данных для обучения и оценки моделей.

2.2 Большие языковые модели в задаче NER

Традиционные подходы к NER обычно основаны на encoder-моделях таких как BERT. Такие модели хорошо подходят для задач классификации токенов или фрагментов текста, однако требуют специализированной архитектуры выходного слоя и отдельной процедуры обучения под конкретный формат разметки.

С развитием больших языковых моделей появилась альтернативная постановка задачи. В этом случае модель получает на вход текст и инструкцию, после чего формирует структурированный ответ, содержащий найденные сущности. В отличие от специализированных архитектур для классификации токенов, такой подход не требует изменения выходного слоя модели и позволяет использовать единый механизм генерации для решения задачи извлечения сущностей. В рамках NER результат генерации может быть представлен в виде JSON-структуры, что делает возможной автоматическую проверку корректности ответа и вычисление метрик качества.

Генеративный подход имеет несколько преимуществ. Во-первых, он позволяет естественно учитывать вложенные сущности, так как модель не ограничена одной меткой на токен. Во-вторых, он делает возможным zero-shot и few-shot применение моделей: даже без дополнительного обучения LLM может попытаться выполнить задачу на основе текстового описания инструкции. В то же время такая постановка создает и новые трудности. Модель может нарушить требуемый формат ответа, сгенерировать невалидный JSON, изменить текст сущности, пропустить редкие типы или добавить сущности, отсутствующие в исходном тексте. Поэтому для успешного применения LLM к NER необходимо не только обучать модель находить сущности, но и контролировать структуру ее ответа.

2.3 Обучение с учителем

Одним из базовых способов адаптации LLM к новой задаче является обучение с учителем, или supervised fine-tuning (SFT). В этом подходе используется набор примеров вида (x, y) , где x обозначает входной запрос, а $y = (y_1, \dots, y_T)$ — эталонный ответ. Для

задачи NER входной запрос содержит инструкцию и текст, а целевой ответ представляет собой список найденных сущностей в заранее заданном формате.

Языковая модель задает условное распределение вероятностей

$$\pi_{\theta}(y \mid x) = \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t}),$$

где θ — параметры модели, а $y_{<t}$ — уже сгенерированная часть ответа. При SFT параметры модели оптимизируются путем минимизации отрицательного логарифма правдоподобия эталонного ответа:

$$\mathcal{L}_{SFT}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}) \right].$$

Иными словами, модель обучается повышать вероятность тех токенов, которые присутствуют в эталонном ответе.

Для задачи NER метод SFT позволяет напрямую обучать модель соблюдать требуемую структуру ответа: использовать фиксированный набор типов сущностей, сохранять JSON-формат и извлекать вложенные сущности. Однако такая постановка имеет существенное ограничение. Оптимизация по кросс-энтропии требует воспроизводить конкретный эталонный ответ, но не учитывает прямую итоговую метрику качества. Например, две генерации могут отличаться только порядком сущностей или содержать частично правильный список объектов, однако функция потерь будет оценивать их на уровне токенов, а не на уровне структуры ответа.

Кроме того, сильная адаптация к узкому набору данных может приводить к переобучению. В этом случае модель повышает качество на целевой задаче, но теряет часть общих способностей, полученных на предыдущих этапах обучения. Такая деградация может проявляться в снижении качества на независимых бенчмарках, связанных с общими знаниями, следованием инструкциям, переводом, программированием или работой с длинным контекстом. Поэтому в настоящей работе SFT рассматривается как важный базовый метод, но его результаты дополнительно сопоставляются с методами обучения с подкреплением.

2.4 Обучение с подкреплением

Обучение с подкреплением рассматривает модель как стратегию π_{θ} , которая по состоянию выбирает действие. В случае языковой модели состоянием можно считать те-

кущий контекст генерации, а действием — выбор следующего токена. В результате последовательного выбора токенов модель порождает полный ответ y на входной запрос x .

Цель обучения с подкреплением состоит в максимизации ожидаемой награды:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [R(x, y)],$$

где $R(x, y)$ — функция награды, оценивающая качество ответа y для запроса x .

В классическом алгоритме RLHF сначала выполняется этап SFT, затем обучается отдельная модель награды $r_\psi(x, y)$ по данным пользовательских предпочтений, после чего языковая модель оптимизируется с помощью алгоритма обучения с подкреплением [1]. Если для одного запроса x имеются два ответа y_w и y_l , где y_w является предпочтительным, а y_l нежелательным, модель награды может обучаться по функции потерь

$$\mathcal{L}_{RM}(\psi) = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\psi(x, y_w) - r_\psi(x, y_l))],$$

где σ — сигмоида. Такая функция потерь увеличивает разность наград между предпочтительным и нежелательным ответами.

После обучения модели награды оптимизируется сама языковая модель. При этом обычно вводится штраф за сильное отклонение от исходной или SFT-модели π_{ref} . Это необходимо для стабилизации обучения и предотвращения неконтролируемого изменения поведения модели. Соответствующая целевая функция может быть записана в виде

$$J_{RLHF}(\theta) = \mathbb{E}_{x, y \sim \pi_\theta} [r_\psi(x, y) - \beta D_{KL}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))],$$

где $\beta > 0$ задает силу регуляризации. В оригинальной работе для оптимизации такой цели часто используется PPO. В этом алгоритме вводится отношение вероятностей

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)},$$

где s_t — состояние на шаге t , a_t — выбранный токен, а $\pi_{\theta_{\text{old}}}$ — стратегия до обновления. PPO ограничивает величину обновления с помощью функции усечения $\text{clip}()$:

$$\mathcal{L}_{PPO}(\theta) = -\mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)A_t)],$$

где A_t — оценка advantage-функции, а ε — гиперпараметр, ограничивающий шаг обновления.

Несмотря на эффективность RLHF, его применение требует значительных ресурсов. Необходимо собирать данные предпочтений, обучать отдельную модель награды, поддерживать value-модель и обеспечивать устойчивость RL-оптимизации. В задачах, где качество ответа может быть вычислено автоматически, такая схема может быть упрощена. Вместо обучения модели награды можно задать явную функцию $R(x, y)$, основанную на проверке результата генерации.

Такая постановка называется RLVR (Reinforcement Learning with Verifiable Rewards, обучение с подкреплением с проверяемой наградой). В математических задачах награда может определяться правильностью финального ответа, в программировании — прохождением тестов, а в задачах структурированного извлечения информации — совпадением предсказанной структуры с эталонной разметкой. Для NER это означает, что ответ модели можно проверить на соответствие JSON-схеме и сравнить с эталонным списком сущностей с помощью стандартных метрик классификации. Поэтому задача извлечения именованных сущностей естественным образом подходит для применения методов RLVR.

2.5 Group Relative Policy Optimization

Одним из современных методов обучения с подкреплением для LLM является Group Relative Policy Optimization (GRPO), предложенный в работе DeepSeekMath [3]. Метод был разработан как более вычислительно эффективная альтернатива PPO для обучения больших языковых моделей.

В классическом PPO требуется отдельная value-модель, оценивающая ожидаемую награду состояния. Для крупных LLM это приводит к существенным вычислительным затратам и увеличению потребления памяти. В GRPO вместо обучения value-модели используется относительное сравнение нескольких ответов, сгенерированных для одного и того же запроса.

Пусть для входного запроса x модель π_θ генерирует группу из G ответов:

$$\{y_1, y_2, \dots, y_G\}.$$

Для каждого ответа вычисляется награда

$$r_i = R(x, y_i).$$

Вместо абсолютной оценки качества используется относительное преимущество ответа внутри группы. Для этого награды нормализуются:

$$A_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})},$$

где

$$\mathbf{r} = (r_1, \dots, r_G).$$

Таким образом, advantage-функция определяется не через отдельную value-модель, а через относительное положение ответа внутри группы генераций.

Как и в PPO, вводится отношение вероятностей:

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}.$$

Тогда функция потерь GRPO записывается в виде

$$\mathcal{L}_{GRPO}(\theta) = -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \sum_t \min(\rho_{i,t}(\theta) A_i, \text{clip}(\rho_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) A_i) \right].$$

Как и в PPO, операция clip ограничивает величину обновления стратегии и предотвращает слишком резкие изменения распределения модели между итерациями обучения. Дополнительно в функцию оптимизации обычно включается штраф KL-дивергенции относительно опорной модели:

$$D_{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}),$$

что позволяет сохранять поведение модели близким к исходному и предотвращает деградацию качества генерации.

Основное преимущество GRPO заключается в отсутствии необходимости обучать отдельную value-модель. Это уменьшает вычислительные затраты и упрощает процесс оптимизации. Кроме того, относительная нормализация награды внутри группы делает обучение менее чувствительным к масштабу функции награды.

В работе DeepSeekMath метод GRPO использовался для обучения математическим рассуждениям, где награда определялась правильностью финального ответа. В дальнейшем аналогичный подход стал применяться и в других задачах с проверяемой наградой, включая программирование, логический вывод и структурированную генерацию.

Для задачи извлечения именованных сущностей использование GRPO представляет большой потенциал. В отличие от стандартного SFT, модель оптимизируется не относительно фиксированного эталонного ответа, а относительно итоговой метрики качества.

Это позволяет напрямую учитывать корректность JSON-структуры, полноту извлечения сущностей, качество работы с редкими типами и другие характеристики ответа. Таким образом, задача NER естественным образом формулируется как задача обучения с проверяемой наградой.

2.6 DAPO

Одной из работ, развивающих идеи GRPO, является DAPO [4]. Этот метод рассматривает практические трудности, возникающие при обучении LLM с помощью алгоритмов обучения с подкреплением. Авторы отмечают, что простое применение GRPO может приводить к неустойчивой оптимизации: модель может слишком быстро уменьшать разнообразие ответов, переусиливать отдельные шаблоны генерации или получать слабый обучающий сигнал на примерах, где все сгенерированные ответы имеют одинаковую награду.

В GRPO для одного запроса генерируется группа ответов, после чего награда каждого ответа нормализуется относительно остальных. Если все ответы в группе имеют одинаковое качество, то относительные преимущества оказываются близкими к нулю. В этом случае пример почти не влияет на обновление параметров модели. DAPO обращает внимание на то, что при обучении полезны прежде всего такие запросы, на которых модель порождает ответы разного качества. Именно они дают различимый сигнал о том, какие варианты генерации следует усиливать, а какие подавлять.

Одним из элементов DAPO является динамическая фильтрация примеров. Пусть для запроса x сгенерирована группа ответов $y_1, \dots, y_G$ с наградами $r_1, \dots, r_G$. Если все награды одинаковы,

$$r_1 = r_2 = \dots = r_G,$$

то такой запрос не содержит относительного обучающего сигнала. В этом случае его можно исключить из текущего шага оптимизации. Напротив, если среди ответов есть как успешные, так и неуспешные, то пример является информативным для обучения.

Другим важным элементом является контроль длины ответа. В задачах рассуждения модель может начать порождать чрезмерно длинные ответы, если это косвенно связано с получением более высокой награды. Для борьбы с этим DAPO вводит штраф награды для слишком длинных ответов. В общем виде такую модификацию можно

записать как

$$\tilde{R}(x, y) = R(x, y) - \lambda \cdot \max(0, |y| - L),$$

где $|y|$ — длина ответа, L — допустимый порог длины, а λ — коэффициент штрафа. Такая поправка не меняет постановку задачи, но ограничивает нежелательную стратегию получения награды за счет избыточной длины генерации.

DAPO также рассматривает изменение функции потерь на уровне токенов. В стандартной формулировке вклад ответа часто усредняется по его длине, что может менять относительный вес коротких и длинных генераций. Для задач, где длина ответа существенно варьируется, это может влиять на устойчивость обучения. Поэтому авторы предлагают более аккуратную нормализацию вклада токенов, чтобы оптимизация лучше соответствовала награде, заданной на уровне всей последовательности.

Для настоящей работы идеи DAPO важны прежде всего методологически. Они показывают, что при применении GRPO недостаточно только задать функцию награды: необходимо также учитывать информативность запросов, распределение наград внутри группы, длину ответа и устойчивость оптимизации. В задаче NER аналогичные проблемы также возникают. Например, если все сгенерированные ответы невалидны или, наоборот, почти полностью совпадают с эталоном, то такой пример дает слабый относительный сигнал. Кроме того, длина ответа связана с числом извлеченных сущностей, поэтому некорректная нормализация может повлиять на баланс между точностью и полнотой.

2.7 Другие модификации GRPO

Помимо DAPO, в сообществе рассматриваются и другие направления анализа и улучшения GRPO. В работе Dr. GRPO [5] исследуются смещения, возникающие из-за нормализации награды и способа учета длины ответа. Авторы обращают внимание на то, что в GRPO награда задается на уровне всей последовательности, тогда как оптимизация проводится на уровне отдельных токенов. Поэтому способ распределения общей награды по токенам может влиять на итоговое поведение модели.

Пусть ответ y имеет длину $|y|$, а его награда равна $R(x, y)$. При усреднении вклада

по токенам оптимизируемая величина фактически зависит от множителя

$$\frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

Такой подход может создавать смещение, связанное с длиной ответа. В одних случаях он ослабляет вклад длинных ответов, в других — может приводить к нежелательной стратегии изменения длины генерации. Для задач структурированного вывода это существенно: в NER длина ответа определяется не только стилем модели, но и числом найденных сущностей. Следовательно, изменение длины ответа может отражать как полезное повышение полноты, так и ошибочное добавление лишних сущностей.

В работе Dr. GRPO подчеркивается, что при использовании GRPO необходимо аккуратно выбирать нормализацию награды и способ вычисления функции потерь. Для настоящей работы этот вывод важен при анализе поведения моделей: рост качества на NER должен рассматриваться совместно с проверкой числа извлеченных сущностей, валидности ответа и деградации на независимых тестовых наборах.

Другое направление связано с повышением вычислительной эффективности GRPO. В работе GRESO [6] рассматривается проблема дороговизны обучения, связанная с необходимостью генерировать несколько ответов на каждый запрос. Если значительная часть запросов не дает полезного различия между ответами, то вычислительные ресурсы расходуются неэффективно. Поэтому предлагается заранее отбирать более информативные запросы или состояния генерации, на которых модель с большей вероятностью получит различающиеся награды.

С точки зрения общей схемы обучения это можно описать как переход от равномерного выбора запросов

$$x \sim \mathcal{D}$$

к выбору из распределения, зависящего от ожидаемой информативности:

$$x \sim q(x),$$

где $q(x)$ повышает вероятность выбора тех примеров, которые дают более полезный обучающий сигнал. Такой подход не меняет целевую задачу, но позволяет уменьшить число неинформативных генераций.

Для задачи NER эта идея также имеет естественную интерпретацию. Не все документы одинаково полезны для обучения: одни содержат большое число разнообразных

сущностей, другие — только несколько простых упоминаний или вообще не создают трудностей для модели. Поэтому методы отбора информативных примеров могут быть полезны при дальнейшем развитии данной работы, особенно если обучение проводится на больших корпусах или требует большого числа генераций на каждый запрос.

Таким образом, современные модификации GRPO показывают, что качество RL-обучения LLM определяется не только выбором базового алгоритма, но и деталями его применения: нормализацией награды, учетом длины ответа, отбором информативных примеров и устойчивостью обновления стратегии. Эти наблюдения важны для адаптации GRPO к задаче извлечения именованных сущностей, где награда является составной и зависит как от формата ответа, так и от качества извлеченной структуры.

3 Постановка задачи

Целью данной работы является исследование методов обучения с подкреплением для адаптации LLM к задаче извлечения именованных сущностей на русском языке. В качестве базовой модели используется Qwen3-4B, а основным набором данных является NEREL, содержащий разметку именованных сущностей, отношений и событий, включая вложенные сущности.

В работе задача NER формулируется как порождение структурированного ответа: модель получает текст и инструкцию, задающую допустимые типы сущностей и требуемый формат ответа.

Для каждого текста задан эталонный список сущностей

$$E^* = \{e_1^*, e_2^*, \dots, e_n^*\},$$

где каждая сущность описывается типом и текстовым фрагментом. После генерации ответ модели преобразуется в список предсказанных сущностей

$$\hat{E} = \{\hat{e}_1, \hat{e}_2, \dots, \hat{e}_m\}.$$

Качество ответа определяется сравнением множеств $\hat{E}$ и E^* , а также проверкой корректности формата ответа.

Основная исследовательская гипотеза состоит в том, что обучение с подкреплением с автоматически вычисляемой наградой позволяет адаптировать LLM к задаче NER без существенной деградации общих способностей модели. Для проверки этой гипотезы в работе сравниваются два подхода: дообучение с учителем и обучение методом GRPO.

Для оценки моделей используются две группы метрик. Первая группа относится непосредственно к задаче NER, вторая включает независимые тестовые наборы из фреймворка LLMTF Open, позволяющие оценить изменение общих способностей модели после дообучения. Такое разделение необходимо, поскольку улучшение на целевой задаче само по себе не гарантирует сохранения универсальных возможностей LLM.

Таким образом, в работе решаются следующие задачи:

1. подготовить данные NEREL для обучения LLM;
2. реализовать обучение модели Qwen3-4B методом SFT;

3. реализовать обучение модели Qwen3-4B методом GRPO с составной функцией награды;
4. провести оценку качества моделей на задаче NER;
5. провести комплексную оценку моделей на независимых бенчмарках LLMTE Open;
6. провести анализ результатов.

4 Практическая часть

4.1 Общая схема экспериментов

Практическая часть работы включает полный цикл адаптации модели Qwen3-4B к задаче извлечения именованных сущностей: подготовку данных, обучение моделей, запуск генерации на тестовых примерах и последующую оценку качества.

Эксперименты были организованы как сравнение двух подходов к обучению. Первый подход — SFT — использует эталонные ответы из обучающей выборки. Вторым подходом — GRPO — используется автоматически вычисляемая награда, зависящая от качества сгенерированного ответа. Такое сравнение позволяет отдельно оценить влияние прямого обучения на размеченных примерах и обучения по метрике качества.

Все основные этапы были реализованы в виде отдельных скриптов. Предобработка исходного набора данных выполняет извлечение текстов и сущностей, фильтрацию части сэмплов и преобразование разметки к единому представлению. Далее формируются два типа обучающих выборок: для SFT сохраняется эталонный ответ модели, а для GRPO сохраняется разметка, необходимая для вычисления награды во время обучения.

Обучение SFT и GRPO проводилось с использованием LoRA-адаптации. Такой подход позволил сократить число обучаемых параметров и проводить серию экспериментов без полного переобучения модели.

Для каждой обученной модели выполнялся автоматизированный запуск генерации на тестовых наборах. Оценка на NEREL проводилась отдельными скриптами, вычисляющими метрики качества извлечения сущностей и корректности формата ответа. Дополнительно модели запускались в инфраструктуре LLMTF Open через API-совместимый интерфейс vLLM, что позволяло проводить единообразную оценку различных конфигураций SFT и GRPO на внешних тестовых наборах. Таким образом, экспериментальная схема была направлена не только на максимизацию качества на целевой задаче, но и на анализ побочных эффектов адаптации.

4.2 Подготовка данных и формирование запросов

Исходный набор данных NEREL содержит тексты документов и разметку сущностей, отношений и событий. Каждая сущность задается типом, текстовым фрагментом и позицией в исходном документе. Позиции использовались на этапе предобработки для восстановления порядка сущностей в тексте, однако в эталонном ответе модели сохранялись только тип сущности и соответствующий текстовый фрагмент.

На этапе предобработки данные приводились к единому внутреннему формату. Из исходной разметки извлекались сущности, после чего выполнялась фильтрация. В процессе подготовки документов текст разделялся по символам переноса строки на отдельные фрагменты, используемые как независимые обучающие примеры. Поэтому разрывные сущности удалялись из выборки, поскольку отдельные части одной сущности могли попасть в разные фрагменты текста. Оставшиеся сущности сортировались по позиции в документе, что позволяло сформировать эталонный ответ в том же порядке, в котором сущности встречаются в тексте.

На вход модели подавалась инструкция, содержащая описание задачи, список допустимых типов сущностей и требование вернуть ответ в строгом JSON-формате. Пользовательская часть запроса содержала текст, из которого необходимо извлечь сущности. Ответ ассистента представлял собой JSON-список элементов вида

$$[\text{TYPE}, \text{entity text}],$$

где **TYPE** — один из допустимых типов сущностей, а **entity text** — фрагмент исходного текста.

Таким образом, эталонный ответ для текста x можно представить как упорядоченный список

$$E^*(x) = [(c_1, s_1), (c_2, s_2), \dots, (c_n, s_n)],$$

где c_i обозначает тип сущности, а s_i — соответствующий текстовый фрагмент. Порядок элементов в списке соответствует порядку появления сущностей в тексте.

Отдельное внимание уделялось формулировке инструкции. Модель должна была не только найти сущности, но и соблюдать несколько ограничений: использовать только заданные типы, не добавлять поясняющий текст вне JSON, сохранять исходное написание сущностей и учитывать вложенные сущности. Последнее требование существенно

для NEREL, поскольку один и тот же участок текста может входить в несколько размеченных объектов.

На основе подготовленных данных формировались две обучающие выборки. Для SFT каждый пример содержал полный диалог: системную инструкцию, пользовательский запрос с текстом и эталонный ответ ассистента. Для GRPO пример содержал инструкцию и текст, а эталонная разметка сохранялась отдельно и использовалась во время обучения для вычисления награды по сгенерированному ответу.

4.3 Обучение методом SFT

Первым способом адаптации модели к задаче NER было дообучение с учителем. Для этого использовалась выборка, в которой каждый пример представлял собой диалог из системной инструкции, пользовательского запроса с текстом и эталонного ответа ассистента. Эталонный ответ содержал список сущностей в JSON-формате. При обучении функция потерь вычислялась только по токенам ответа ассистента.

Пусть обучающий пример имеет вид (x, y) , где x — инструкция вместе с исходным текстом, а $y = (y_1, \dots, y_T)$ — эталонный JSON-ответ. Тогда функция потерь для одного примера записывается как

$$\mathcal{L}_{SFT}(\theta; x, y) = - \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

Обучение проводилось с использованием LoRA-адаптации. В экспериментах с SFT низкоранговые матрицы добавлялись к проекциям attention-механизма

$$q_proj, k_proj, v_proj, o_proj,$$

а также к линейным слоям MLP-блоков

$$gate_proj, up_proj, down_proj.$$

Базовые параметры Qwen3-4B при этом не обновлялись. Обучаемыми являлись только дополнительные низкоранговые матрицы, что позволило снизить вычислительные затраты.

В основной конфигурации SFT использовался размер батча 2 и накопление градиента в течение 8 шагов. Модель обучалась 3 эпохи со скоростью обучения $2 \cdot 10^{-5}$,

weight decay 0.05 и warmup на 16 шагов. В качестве оптимизатора использовался `adamw_torch_fused`. Максимальная длина входной последовательности составляла 2048 токенов. Обучение выполнялось без квантизации.

Помимо основной SFT-модели, была обучена отдельная группа моделей с более агрессивными настройками адаптации к NEREL. Эти эксперименты использовались как диагностические: они должны были показать, какие способности начинают ухудшаться при чрезмерной специализации модели на задаче извлечения сущностей.

4.4 Обучение методом GRPO

Вторым способом адаптации модели было обучение методом GRPO. В отличие от SFT, где модель обучается воспроизводить эталонный ответ, в GRPO оптимизация проводится по награде, вычисляемой для сгенерированного ответа. Для задачи NER это позволяет напрямую учитывать не только совпадение с разметкой, но и корректность формата ответа.

Для GRPO использовалась отдельная обучающая выборка. Каждый пример содержал системную инструкцию, пользовательский запрос с текстом и эталонную разметку, сохранённую в поле `gold_batch`. Эталонный ответ не включался в диалог как сообщение ассистента, поскольку модель должна была самостоятельно сгенерировать список сущностей, а затем получить награду за результат генерации.

Для одного входного запроса x модель порождала группу из G ответов:

$$y_1, y_2, \dots, y_G,$$

после чего для каждого ответа вычислялась награда

$$r_i = R(x, y_i).$$

В экспериментах использовалось значение $G = 8$.

Функция награды была реализована как составная величина:

$$R(x, y) = 0.05R_{\text{json}}(x, y) + 0.05R_{\text{schema}}(x, y) + 0.85R_{\text{macroF1}}(x, y) + 0.05R_{\text{order}}(x, y).$$

Компонент R_{json} равен 1, если ответ модели удалось разобрать как JSON-список, и 0 в противном случае. Если ответ не является корректным JSON, итоговая награда полагается равной нулю.

Компонент R_{schema} оценивает соответствие ответа требуемой схеме. Каждый элемент ответа должен быть списком из двух строк: типа сущности и текстового фрагмента. Тип сущности должен принадлежать фиксированному множеству допустимых тегов NEREL, а текст сущности не должен быть пустым. Значение R_{schema} вычисляется как доля элементов ответа, удовлетворяющих этим условиям.

Основным компонентом награды является R_{macroF1} . Для его вычисления предсказанные и эталонные сущности приводятся к парам вида

$$(c, s),$$

где c — тип сущности, а s — нормализованный текстовый фрагмент. Далее для каждого типа сущностей вычисляются значения precision, recall и F1, после чего берётся среднее значение F1 по типам. Использование macro-F1 делает награду более чувствительной к качеству работы с редкими классами сущностей.

Компонент R_{order} отвечает за сохранение порядка сущностей в тексте. Он вычисляется на основе длины наибольшей общей подпоследовательности между эталонным и предсказанным списками сущностей:

$$R_{\text{order}} = \frac{\text{LCS}(E^*, \hat{E})}{|E^*|},$$

где E^* — эталонный список сущностей, а $\hat{E}$ — список сущностей, извлечённых из ответа модели. Этот компонент имеет небольшой вес, но дополнительно поощряет модель возвращать сущности в порядке их появления в тексте.

Обучение GRPO проводилось с использованием LoRA, низкоранговые матрицы добавлялись ко всем линейным слоям модели.

В основной конфигурации GRPO использовался размер батча 4 и накопление градиента в течение 16 шагов. Скорость обучения составляла $2 \cdot 10^{-5}$. Для каждого запроса генерировалось 8 ответов; генерация выполнялась с температурой 0.7 и параметром $top_p = 0.9$ в режиме `/no_think`. Максимальная длина входной последовательности составляла 2048 токенов, максимальная длина ответа — 256 токенов. Обучение проводилось с использованием vLLM в режиме совместного размещения с обучением; доля видеопамяти, выделяемая vLLM, составляла 0.65.

Таким образом, GRPO-обучение в данной работе было сведено к оптимизации модели относительно явно заданной функции качества NER. В отличие от SFT, такой подход

не требует задавать единственный эталонный путь генерации на уровне токенов, а позволяет оценивать ответ как структуру: с учётом валидности формата, корректности схемы, совпадения сущностей и их порядка.

4.5 Оценка качества на задаче NER

Для оценки качества извлечения именованных сущностей использовались метрики precision, recall, micro-F1 и macro-F1. Сравнение выполнялось между эталонным множеством сущностей

$$E^*$$

и множеством сущностей

$$\hat{E},$$

извлеченных из ответа модели.

Сущность считалась предсказанной корректно, если совпадали ее тип и текстовый фрагмент. Далее вычислялись значения Precision, Recall и F1-меры.

В работе использовались как micro-F1, так и macro-F1. Метрика micro-F1 вычисляется глобально по всем сущностям и сильнее зависит от наиболее частых классов. Macro-F1 вычисляется как среднее значение F1 по типам сущностей и поэтому более чувствительна к качеству работы с редкими категориями. Поскольку набор NEREL содержит классы с существенно различающейся частотой, macro-F1 использовалась как основная метрика качества.

Помимо стандартных метрик извлечения сущностей дополнительно вычислялись доля валидных JSON-ответов и доля ответов, полностью совпадающих с эталонной разметкой. Первая метрика характеризует способность модели соблюдать требуемый формат генерации, а вторая — качество полного восстановления структуры ответа без частичных совпадений.

4.6 Оценка с использованием LLMTF Open

Для комплексной оценки моделей использовался фреймворк LLMTF Open. Его роль в том, чтобы измерять не только качество на целевой задаче NER, но и изменение других способностей модели после дообучения. Оценка выполнялась

через API-совместимый интерфейс vLLM. Для всех моделей использовался единый конфигурационный файл с одинаковым набором задач и параметрами генерации.

Используемые тестовые наборы:

Назначение бенчмарков	Названия бенчмарков
Извлечение именованных сущностей	NEREL-(json), NEREL-BIO-(json), patient_queries_ner-(json), collection3-(json); NEREL-(in-place), NEREL-BIO-(in-place), patient_queries_ner-(in-place), collection3-(in-place)
Вопросно-ответные задачи с контекстом (RAG)	rubqqa, rubqqa_data_first, rus_tydiqa, rus_tydiqa_data_first; sberquadqa, sberquadqa_data_first, rus_xquadqa, rus_xquadqa_data_first
Общие знания и задачи с выбором ответа	ruMMLU zero-shot, enMMLU zero-shot, ruMMLU few-shot, enMMLU few-shot; moviesmc, musicmc, lawmc, booksmc
Следование инструкциям	ruIFEval, enIFEval
Работа с длинным контекстом	ru_sci_abstract_retrieval, ru_babilong_qa1-qa5, ru_tpo, ru_qasper
Машинный перевод	flores_ru_en, flores_en_ru
Суммаризация	Gazeta, TreewayAbstractive
Классификация текста	sentiment_kinopoisk, RuOpinionNE, RuOpinionNE_simple, RuCoLA

Таблица 1: Группы бенчмарков LLMTF Open, использованные в экспериментах.

Группа NER-бенчмарков использовалась для проверки переноса навыка извлечения сущностей за пределы обучающей постановки. В конфигурации присутствовали два формата: `ner_json`, где модель должна вернуть список сущностей в JSON-структуре, и `ner_in-place`, где разметка выполняется непосредственно внутри исходного текста. Оба режима запускались в few-shot постановке с тремя примерами.

Вопросно-ответные RAG-задачи проверяли способность модели использовать предо-

ставленный контекст при ответе на вопрос. В эту группу вошли наборы на основе RuBQ, TyDiQA, SberQuAD и XQuAD. Для них использовались длинные входные запросы до 16000 токенов. Эти бенчмарки важны как проверка того, сохраняет ли модель способность извлекать релевантную информацию из контекста после дообучения на NER.

Группа MMLU и Shlepa использовалась для оценки общих знаний. Русскоязычная и англоязычная версии MMLU запускались как в zero-shot, так и в few-shot режиме. Наборы Shlepa содержат вопросы с выбором ответа из нескольких предметных областей, включая фильмы, музыку, право и книги. Снижение качества на этой группе может указывать на ухудшение знаний модели после адаптации.

Бенчмарки ruIFEval и enIFEval проверяли следование инструкциям. Эти задачи требуют от модели соблюдать ограничения, заданные в запросе.

Задачи семейства LIBRA использовались для оценки работы с длинным контекстом. В эту группу вошли поиск информации в научных аннотациях, поднаборы ruBABILong QA1–QA5, ruTPO и ruQasper. Максимальная длина входного запроса для этих задач составляла 32000 токенов. Эти бенчмарки позволяют проверить, не ухудшается ли способность модели работать с большими текстами.

Дополнительные группы включали перевод, суммаризацию, анализ тональности и лингвистическую корректность. Перевод оценивался на русско-английских и англо-русских данных FLORES. Суммаризация запускалась на наборах Gazeta и TreewayAbstractive. Для классификационных задач использовались наборы анализа тональности и RuCoLA.

4.7 Результаты экспериментов

4.7.1 Проведенные серии экспериментов

В ходе работы были обучены серии моделей SFT и GRPO с различными конфигурациями. Для обеих групп экспериментов исходной моделью служила Qwen3-4B.

В серии SFT-экспериментов были обучены модели на основной конфигурации, а также на долях 20%, 50% и 100% NEREL. Дополнительно проверялось влияние увеличения $learning_rate$ $5 \cdot 10^{-5}$ и 10^{-4} и уменьшения $lora_alpha = 16$. Отдельные конфигурации с 50% или 100% NEREL и увеличенным $learning_rate$ рассматривались как намеренно переобученные SFT-модели.

В серии GRPO экспериментов были рассмотрены доли 10%, 20%, 50% и 100% NEREL, *learning_rate* 10^{-5} , $5 \cdot 10^{-5}$ и 10^{-4} , значения *gradient_accumulation_steps* 8, 16 и 32, а также температура 0.7 и 1.0. Кроме того, проводились эксперименты с различными весами компонент награды: с близкими весами для всех компонент и с увеличенным вкладом проверки формата ответа.

Результаты всех запусков фиксировались по метрикам качества NER: доле валидных JSON-ответов, доле полных совпадений с эталоном, micro-precision, micro-recall, micro-F1, macro-precision, macro-recall и macro-F1. Для дальнейшего анализа использовались не все обученные модели, а несколько характерных конфигураций, отражающих основные режимы поведения модели.

4.7.2 Итоговое сравнение на LLMTE

Для подробного сравнения были выбраны четыре модели. Такой выбор позволяет сопоставить исходное качество модели, результат обучения с подкреплением, результат стандартного SFT и поведение SFT-модели при чрезмерной специализации на NEREL.

Модель	Описание
baseline_qwen3_4b	Исходная модель Qwen3-4B.
qwen3_4b_20260521_200100	Лучшая GRPO-модель среди основных запусков. Обучалась на 100% NEREL с увеличенным <i>learning_rate</i> $5 \cdot 10^{-5}$.
qwen3_4b_sft_20260330_103912	Лучшая SFT-модель. Обучалась на 50% NEREL в основной SFT-конфигурации.
qwen3_4b_sft_20260517_031135	Переобученная SFT-модель. Обучалась на 50% NEREL с увеличенным <i>learning_rate</i> 10^{-4} .

Таблица 2: Модели, выбранные для подробного сравнения.

4.7.3 Анализ полученных результатов

По агрегированным результатам видно, что обе специализированные модели — SFT и GRPO — улучшают качество на группе NER по сравнению с базовой моделью. При этом GRPO показывает наибольшее значение: 0.421 (+37%) против 0.402 (+30%) у лучшей SFT-модели и 0.307 у исходной Qwen3-4B. Это означает, что обучение с подкрепле-

Модель	NER	RAG / QA	MMLU / Shlepa	IFEval
Baseline	0.307	0.691	0.556	0.738
GRPO	0.421	0.698	0.543	0.743
SFT	0.402	0.715	0.544	0.747
SFT overfit	0.335	0.692	0.533	0.699

Модель	LIBRA	Перевод	Суммаризация	Классификация	Среднее
Baseline	0.566	0.556	0.212	0.348	0.516
GRPO	0.650	0.583	0.226	0.332	0.544
SFT	0.561	0.568	0.229	0.348	0.537
SFT overfit	0.609	0.577	0.214	0.313	0.511

Таблица 3: Агрегированные результаты выбранных моделей на LLMTF Open.

нием лучше перенесло навык извлечения сущностей на совокупность NER-бенчмарков LLMTF, включая форматы и датасеты, отличающиеся от основной обучающей постановки.

На группе MMLU / Shlepa наблюдается снижение как у SFT, так и у GRPO.

Отличие GRPO проявляется в других группах задач. Модель, обученная GRPO, показывает лучший результат на LIBRA, улучшает перевод, суммаризацию и повышает качество следования инструкциям (0.743 против 0.738). Это указывает, что после GRPO-адаптации модель не только улучшила качество на NER, но и сохранила или улучшила часть внешних способностей.

Переобученная SFT-модель демонстрирует иное поведение. Несмотря на специализированное обучение на NEREL, её агрегированный результат на NER возрастает лишь до 0.335, то есть оказывается ниже, чем у обычной SFT-модели и GRPO. Одновременно ухудшаются IFEval, MMLU / Shlepa и классификационные задачи. Это подтверждает, что чрезмерная настройка на обучающий набор не обязательно улучшает перенос навыка на другие NER-бенчмарки и может сопровождаться ухудшением общих навыков.

Дополнительно следует отметить, что GRPO-модель показала лучшее значение средней оценки среди всех рассмотренных конфигураций. Среднее значение на LLMTF Open для неё составило 0.544, тогда как для лучшей SFT-модели — 0.537, а для базовой

модели — 0.516.

Таким образом, основной вывод состоит в следующем: GRPO сильнее улучшает обобщённое качество извлечения сущностей на разных NER-бенчмарках LLMTF и при этом сохраняет или улучшает несколько других групп способностей, вызывая сопоставимую с SFT деградацию знаний.

5 Инструментальные средства

Программная часть работы была реализована на языке Python. Для обучения моделей использовались библиотеки `transformers`, `datasets` и `peft`.

SFT-обучение проводилось с использованием библиотек Hugging Face, а GRPO-обучение — с использованием фреймворка `ms-swift`. Для эффективной генерации и запуска оценки применялся `vLLM`.

Комплексная оценка моделей выполнялась с помощью фреймворка `LLMTF Open`. Для подготовки данных, запуска экспериментов и агрегации результатов были реализованы собственные Python-скрипты.

6 Заключение

В работе исследованы методы адаптации больших языковых моделей к задаче извлечения именованных сущностей на русском языке. В качестве базовой модели использовалась Qwen3-4B, а обучение проводилось на наборе данных NEREL в формате структурированного JSON-ответа. Были реализованы и исследованы два подхода к обучению: SFT и обучение с подкреплением методом GRPO. Для GRPO была разработана функция награды, учитывающая различные критерии ответа.

В рамках работы был построен полный экспериментальный пайплайн: подготовка данных из датасета NEREL, обучение моделей, запуск оценки, агрегация результатов и построение графиков для контроля обучения. Дополнительно была проведена комплексная оценка моделей с использованием фреймворка LLMTF Open на широком наборе бенчмарков.

Эксперименты показали, что GRPO позволяет существенно улучшить качество извлечения сущностей на различных NER-бенчмарках. При этом снижение результатов на задачах общих знаний оказалось сопоставимым с SFT, однако GRPO продемонстрировал более сильный перенос навыка NER на новые бенчмарки, а также сохранение и улучшение ряда других способностей модели.

Отдельная серия переобученных SFT-моделей показала, что чрезмерная специализация на NEREL может сопровождаться ухудшением качества на внешних задачах и не гарантирует лучшего переноса навыка извлечения сущностей.

Полученные результаты показывают, что методы обучения с подкреплением являются перспективным направлением адаптации LLM к задачам обработки естественного языка на русском языке. В проведенных экспериментах GRPO продемонстрировал лучшее качество на совокупности всех бенчмарков. Это указывает на то, что RL-подходы способны обучать модель новым специализированным навыкам, экстраполирующимся на альтернативные задачи, и представляют собой сильную альтернативу классическому SFT-дообучению.

Список литературы

- [1] **Ouyang L. et al.** Training language models to follow instructions with human feedback // *arXiv preprint arXiv:2203.02155*. – 2022.
- [2] **Loukachevitch N. et al.** NEREL: A Russian Dataset with Nested Named Entities, Relations and Events // *Proceedings of Recent Advances in Natural Language Processing*. – 2021.
- [3] **Shao Z. et al.** DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models // *arXiv preprint arXiv:2402.03300*. – 2024.
- [4] **Yu D. et al.** DAPO: An Open-Source LLM Reinforcement Learning System at Scale // *arXiv preprint arXiv:2503.14476*. – 2025.
- [5] **Liu X. et al.** Understanding R1-Zero-Like Training: A Critical Perspective // *arXiv preprint arXiv:2503.20783*. – 2025.
- [6] **Zheng H. et al.** Act only when it pays: Efficient reinforcement learning for llm reasoning via selective rollouts // *Advances in Neural Information Processing Systems*. – 2026.
- [7] **Yang A. et al.** Qwen3 Technical Report // *arXiv preprint arXiv:2505.09388*. – 2025.
- [8] **Hu E. J. et al.** LoRA: Low-Rank Adaptation of Large Language Models // *arXiv preprint arXiv:2106.09685*. – 2021.
- [9] **Zheng C. et al.** Stabilizing reinforcement learning with llms: Formulation and practices // *arXiv preprint arXiv:2512.01374*. – 2025.
- [10] **Team Q.** Qwen3. 5-omni technical report // *arXiv preprint arXiv:2604.15804*. – 2026.